Process Migration on a RISC-V Microkernel: Revisiting the Kernel-Resident versus User-Space Verdict

A. S. R. Silva
School of Computing and Engineering
National Institute of Business Management, Colombo, Sri Lanka
in collaboration with Coventry University, United Kingdom

Abstract*—In 1993, Milojicic et al. implemented process migration both inside the Mach kernel and as a user-space server, and reported the user-space design to be roughly an order of magnitude slower under load. That measurement has been cited ever since as grounds for placing distribution mechanism inside the kernel. Three things have changed and the verdict has not been re-examined: microkernel IPC overhead has fallen to within a few per cent of monolithic kernels, checkpoint and restore has reached production in container platforms and automotive operating systems, and RISC-V has become the open instruction set on which current microkernel work is built. We repeat the 1993 comparison on a contemporary RISC-V microkernel. Two builds are produced from one source tree, differing only in whether the migration mechanism is invoked from inside the kernel or from a user-space server, and are instrumented at nine phase boundaries with the RISC-V cycle counter. Across 60 migrations per build and four checkpoint sizes, the user-space to kernel-resident total-latency ratio is 1.00 at 95 per cent confidence, and no phase differs significantly. The in-kernel code attributable to migration is 393 lines, against 56,000 for Mach and 3,020 for the Amoeba process server measured from its own source. We also report a result that does not favour our design: measured against a real Amoeba 5.3 system running its own benchmarks, an inter-process round trip costs 39.7 null system calls on our kernel and 26.0 on Amoeba. We conclude that the performance argument for kernel-resident migration no longer holds, while noting that our IPC path remains the weaker of the two.

***Keywords*—process migration, microkernel, RISC-V, trusted computing base, inter-process communication, operating system architecture**

I. Introduction

Process migration — moving a running process from one machine to another and resuming it — has been implemented repeatedly since the early 1980s [16], [17], and repeatedly abandoned. The canonical survey of the field [2] devotes a section to why, listing both technical and non-technical obstacles. One technical finding has been particularly durable. In 1993, Milojicic et al. [1] implemented migration twice on the same Mach microkernel [4]: once as a user-space migration server, and once inside the kernel using the distributed IPC and distributed memory management subsystems. On Intel 80486 hardware the user-space variant was roughly an order of magnitude slower under load. The conclusion drawn, and cited since, is that distribution mechanism belongs inside the kernel.

That conclusion was reached on 1993 hardware and on a kernel whose IPC was, by the measurements of the day, the slowest of its generation. Three developments since then bear directly on the verdict and have not been used to re-test it. First, microkernel IPC overhead has fallen substantially; recent work places it within a few per cent of monolithic kernels on the same workloads [8], [9]. Second, checkpoint and restore has reached production, in container platforms through CRIU [19] and in automotive operating systems deployed at scale [21]. Third, RISC-V [13] has become the open instruction set on which much current microkernel research is built, and no open-source migration implementation for it exists.

This paper repeats the 1993 experiment on a contemporary RISC-V microkernel. The contribution is deliberately narrow and empirical. We build two kernels from one source tree that differ only in where the migration mechanism is invoked from, instrument both at the same nine phase boundaries with the RISC-V cycle counter, and measure them under identical conditions. We then ask a second question the 1993 work did not: how much kernel code each design costs, which is the question a modern security argument asks [6], [7]. Finally, we obtain a second reference point by booting a real Amoeba 5.3 system and running its own benchmarks against ours.

We report three results.

The architectural placement is performance-neutral. Over 60 migrations per build across four checkpoint sizes, the user-space to kernel-resident total-latency ratio is 1.00 at 95 per cent confidence, and no individual phase differs significantly (Section V-A).

The kernel cost differs by orders of magnitude. The in-kernel code attributable to migration is 393 lines, against 56,000 lines for Mach's DIPC and DMM [3] and 3,020 lines for the Amoeba process server, which we measured directly from the Amoeba source rather than quoting it (Section V-D).

Our IPC path is the weaker one. Measured against a real Amoeba 5.3 system running its own benchmark programs, an inter-process round trip costs 39.7 null system calls on our kernel and 26.0 on Amoeba (Section V-E). We report this

because an architectural recommendation that suppressed it would not be worth making.

II. Background and Related Work

A. First-generation process migration

Migration was prototyped repeatedly on research operating systems for workstation networks. LOCUS [17] and DEMOS/MP [16] moved processes as a system service. The V kernel [18] implemented pre-emptive task migration on top of its message-passing primitives. MOSIX [15] provided transparent, load-triggered migration in a UNIX-compatible system, and Sprite [14] built migration on its distributed file system with explicit attention to the cost of moving open file state. This generation established feasibility but converged on nothing: systems differed in state-transfer strategy, in scheduling policy, and — the question this paper takes up — in where the mechanism should live [2].

Table I places these systems side by side. Two things stand out. Every first-generation implementation put the mechanism in the kernel, so the placement question was not so much settled as never posed; and only the 1993 Mach work built both placements on one kernel and compared them, which is why it is the study this paper repeats.

TABLE I. Representative process-migration implementations

System	Year	Mechanism	State transferred
DEMOS/MP [16]	1983	kernel	process, message channels
LOCUS [17]	1983	kernel	process, open files
MOSIX [15]	1985	kernel	process, file state
V [18]	1988	kernel	address space, pre-copy
Sprite [14]	1991	kernel	process, file state
Mach [1]	1993	both compared	task, memory objects
CRIU [19]	2011–	user	process tree, descriptors
This work	2025	both compared	registers, stack, payload

B. The 1993 Mach measurement

Milojicic et al. [1] built three migration designs on Mach. The comparison that matters here is between the user-space migration server and the kernel-resident implementation built on distributed IPC (DIPC) and distributed memory management (DMM). The later survey [3] gives the sizes of those subsystems: DIPC is over 32,000 lines and DMM 24,000. The user-space server was slower because each operation it performed crossed the kernel boundary, and on Mach that crossing was expensive. The kernel-resident design avoided the crossings by absorbing the mechanism, and the infrastructure it absorbed was large.

It is worth recording how Mach's IPC compared with its contemporaries, because the verdict is a direct consequence of it. Fig. 1 reproduces the cross-system comparison collected by Tanenbaum et al. [10]. Mach's null RPC of 11.0 ms was an order of magnitude slower than Amoeba's 1.1 ms on comparable hardware. The 1993 experiment therefore measured the penalty of crossing a boundary on the system where that crossing was most expensive.

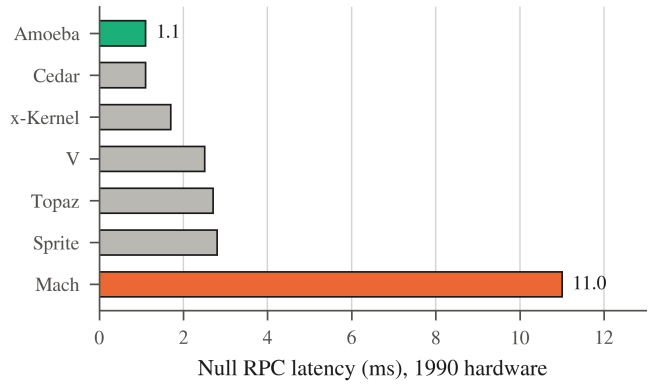


Fig. 1. Null-RPC latency reported in the literature, c. 1990, from [10, Fig. 1]. These are 2–5 MIPS machines and are not comparable with the emulated measurements reported later; they establish the relative standing of the systems at the time.

C. What has changed since

The cost asymmetry that produced the 1993 result was a property of Mach's IPC, not a law of microkernel design. Liedtke [5] showed shortly afterwards that IPC cost is an implementation property, and the L4 family that followed reduced it by more than an order of magnitude. Recent work continues the trend, reporting microkernel performance within a few per cent of monolithic systems [8], [9].

In parallel, the security argument has inverted the sign of the trade. seL4 [6] made the size of the trusted computing base the central design variable, and Biggs et al. [7] argue on CVE evidence that moving code out of the kernel measurably reduces exposure. Code that the 1993 analysis treated as a cost worth paying for speed is now treated as a liability to be minimised. A comparison that reports only latency therefore measures the wrong thing for a contemporary decision, which is why Section V-D reports code size alongside it.

Finally, the workload has changed. Migration in the 1990s competed with simply restarting a stateless job, and usually lost. The workloads that motivate it now — long-running training jobs holding GPU state, and safety-critical automotive partitions [21] — are ones for which restarting is either expensive or not permitted.

D. Migration in production today

The 1999 survey [3] could point to no widely deployed use of process migration. That is no longer the case, though the deployments took a different form from the one the survey anticipated. Checkpoint and restore in userspace [19] is

shipped in mainstream Linux distributions and used by container platforms to suspend, relocate and restart workloads; the unit of migration became the container rather than the process, but the mechanism underneath is the same checkpoint, transfer and restore. Virtual-machine live migration made the same transition a decade earlier at coarser granularity. In embedded systems, partitioned operating systems deployed in automotive electronics [21] relocate workloads across partitions as a fault-containment measure.

This matters for our argument in one specific way. The 1993 comparison was between two designs for a mechanism that nobody was yet deploying, so its verdict was never tested against a real operational requirement. The requirement now exists, and the security properties of the code that implements it are part of what operators are buying — which is why Section V-D measures code size at all.

E. Amoeba as a second reference point

Mach is not the only system against which this comparison can be made, and it has the disadvantage that its line counts must be quoted from the literature. Amoeba [10], [11] is a distributed operating system of the same generation whose source is freely distributed [12], which makes it measurable rather than quotable. It is also the stronger comparison in one respect: as Fig. 1 shows, Amoeba was the IPC performance leader of its generation, so it represents the best case for the design philosophy the 1993 verdict endorsed. We use it in Sections V-D and V-E both for its code size and for a direct performance comparison.

III. Design and Implementation

A. Kernel primitives

The kernel exposes migration through two system calls. sys_checkpoint(pid, buf, len) suspends a task and serialises its state into a caller-supplied buffer; sys_restore(buf, len) instantiates a task from such a buffer and marks it runnable. Nothing about network transport, peer selection, retry, or policy appears in either call. Those two functions, with their header, are the 393 lines of kernel code that Section V-D counts.

B. The checkpoint record

The record is a self-describing binary blob whose layout is given in Table II. It is position-independent in one specific sense that matters for correctness: the saved stack pointer is stored as an offset from the base of the stack image rather than as an absolute address. A restored task is given a fresh stack at a different address, and without the offset encoding the restored register file would point into the source task's address space. This detail cost more debugging time than any other part of the implementation and is the reason the record survives transport to a different machine unchanged.

TABLE II. Checkpoint record layout

Region	Size	Contents
Header	40 B	magic, version, pid, sizes
Registers	112 B	saved integer register file
Stack	8192 B	stack image, SP as offset
Payload	0–64 KB	optional, harness-controlled

C. The two builds

The kernel-resident and user-space builds are produced from the same source tree and selected by a single build flag, which controls one #ifdef in the test harness. Both code paths are present in both binaries; only the path exercised differs. In the kernel-resident build the harness calls the migration routine directly from supervisor mode, modelling the 1993 Mach design. In the user-space build the mechanism is driven by a server that reaches the kernel only through the two system calls above, modelling the policy-mechanism separation of [5] and [6]. Fig. 2 shows the two arrangements.

Building both from one tree removes the most obvious confound in the original comparison, which necessarily compared two separately engineered implementations and so could not fully separate the architectural difference from differences in engineering quality.

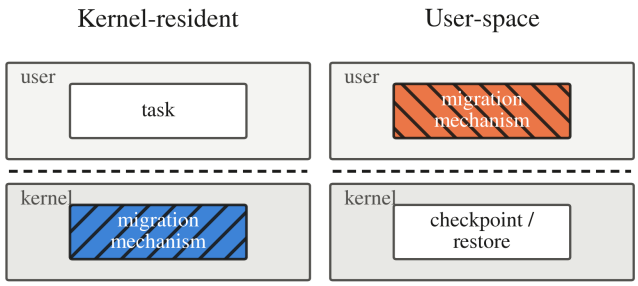


Fig. 2. The two builds. The migration mechanism moves across the privilege boundary; the checkpoint and restore primitives stay in the kernel in both cases. Only the invocation path differs.

D. Cross-node transport

To confirm that the checkpoint record is a sufficient description of a task, a second harness migrates between two emulator instances over real Ethernet frames. The transport is a VirtIO-Net device driven by a modern MMIO driver, carried over a UDP multicast segment on the host. An 8 KB checkpoint fragments into six frames, each carrying a fifteen-byte fragment header giving a magic value, message type, total size, offset and fragment length; the destination reassembles in order and calls sys_restore on completion. This path is not part of the placement comparison. It exists to show that the record survives a real network rather than a shared memory buffer.

IV. Methodology

A. Instrumentation

Both builds are instrumented at nine phase boundaries with a macro that reads the RISC-V unprivileged cycle CSR and emits one line over the serial port. Five metrics are derived: freeze time, serialise, transport, deserialise, and the total from entry to the restored task becoming runnable. The instrumentation is identical in both builds, so any overhead it introduces is common-mode and cancels in the comparison. It is not negligible in absolute terms: the serial write is synchronous, and the sum of the measured phases is smaller than the measured total by the cost of the unmeasured gaps between brackets.

B. Experimental design

Each build performs 60 migrations, cycling through four checkpoint sizes — 8, 12, 24 and 72 KB — giving 15 samples per cell. Sizes are varied by changing only the optional payload region, so the fixed cost of the register file and stack image is constant across cells and the size effect is isolated. We report the arithmetic mean and the 95 per cent confidence interval half-width, computed as $1.96 s/\sqrt{n}$ under the central-limit approximation. We treat two conditions as indistinguishable when their confidence intervals overlap; we do not claim equivalence beyond that, and in particular we do not perform an equivalence test with a pre-specified margin.

C. The Amoeba reference system

For Sections V-D and V-E we boot FSD-Amoeba 2002A [12], the i386 packaging of Amoeba 5.3, under emulation [20] on the same host that runs the RISC-V measurements. The kernel, the FLIP protocol switch, the Bullet file server and the Soap directory server all start normally. Line counts are taken with wc -l over the 2001 kernel source tree shipped with that distribution, comprising 287 source and header files.

Performance figures are produced by Amoeba's own test programs — a null system call benchmark and an RPC client/server pair written by the original authors and shipped precompiled — rather than by anything we wrote, so the comparison is not biased by our choice of benchmark. On our side we added an equivalent pair that issues real user-mode traps rather than direct kernel calls, so that the two measurements describe the same kind of event.

D. Reviving the reference system

Using a 2002 operating system as a live reference point required two obstacles to be cleared, and we record them because they are the reason such comparisons are rarely attempted.

First, Amoeba’s own disk partitioning tool writes a partition entry but does not write the 0xAA55 boot signature into the master boot record. On period hardware the signature was already present, written by DOS; on a blank emulated disk it is not, and Amoeba’s own virtual-disk server then rejects the partition it has just created, reporting an invalid partition table. Writing the two bytes externally resolves it.

Second, the installer cannot read the distribution CD: the IDE driver does not support ATAPI, and the setup scripts contain no reference to a CD-ROM at all. Software is loaded instead from floppy volumes, each a 512-byte volume label followed by a raw slice of a compressed archive. We reconstructed that format from the distribution’s own sources — the label carries a magic number, media and data sizes, volume sequence numbers and a CRC-16/CCITT over the payload seeded with 0xFFFF — and verified the reconstruction by regenerating the shipped volumes byte for byte from their own payloads. That allows arbitrary packages, including the benchmark suite used in Section V-E, to be installed into the running system.

Neither obstacle is a defect of Amoeba as it was used in 2002. Both are artefacts of running period software on emulated hardware, and both are the kind of detail that silently prevents historical systems from being used as reference points. Our scripts for the procedure are published with the rest of the artefact.

E. Reproducibility

Every number in this paper is produced by a scripted pipeline from a clean checkout. The two kernels are built by setting one make variable; the sweep harness boots each under the emulator, captures the serial output, and extracts the instrumented lines into a log; an analysis script groups the lines into per-migration records and computes the statistics reported in Table III. The figures are regenerated from those same logs by a single script, so no figure in this paper contains a hand-entered number. The raw logs are published alongside the source, which allows the analysis to be re-run or replaced without re-running the experiments.

V. Evaluation

A. Architectural placement

Table III gives the per-phase and total costs for both builds. Every phase ratio lies between 0.99 and 1.00, and every confidence interval overlaps. The headline total-latency ratio is 1.00. On this kernel, the placement of the migration mechanism is not a performance question.

TABLE III. Migration cost by phase, kilocycles, mean ± 95% CI half-width, n = 60 per build

Phase	Kernel-res.	User-space	Ratio
Freeze	275.4 ± 9.1	272.8 ± 7.2	0.99
Serialise	1078.7 ± 55.3	1076.7 ± 48.5	1.00
Transport	782.9 ± 15.1	785.9 ± 14.8	1.00
Deserialise	1029.2 ± 70.0	1016.8 ± 59.3	0.99
Total	4061.7 ± 145.5	4061.8 ± 118.2	1.00

Equal means can hide unequal distributions, so Fig. 3 plots the spread rather than repeating the averages. The boxes overlap at every checkpoint size and the medians sit within each other's interquartile ranges. The equality in Table III is not an artefact of averaging over a heavy tail.

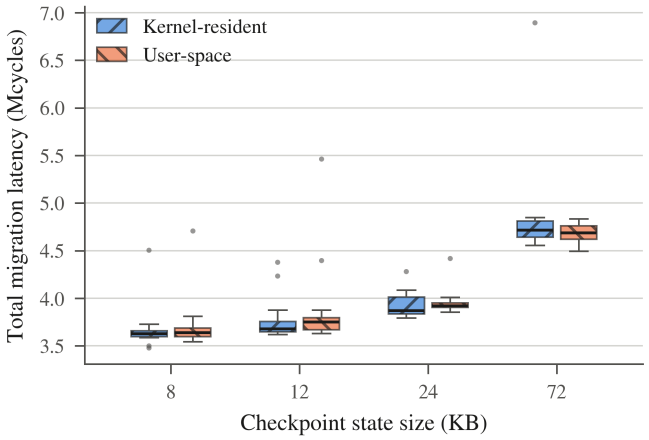


Fig. 3. Total migration latency by checkpoint size. Boxes show the interquartile range, whiskers 1.5 IQR, points outliers. The two builds overlap at every size.

B. Where the time goes

Fig. 4 decomposes the cost. Serialise and deserialise together account for most of it and absorb essentially all of the growth with state size, which is what one expects of phases whose work is proportional to the bytes copied. Freeze is small and flat: suspending a task is a constant-cost operation on a fixed-size task table. Transport is flat because in the local harness it is a memory copy, not a network operation; Section V-F measures what happens when it is not.

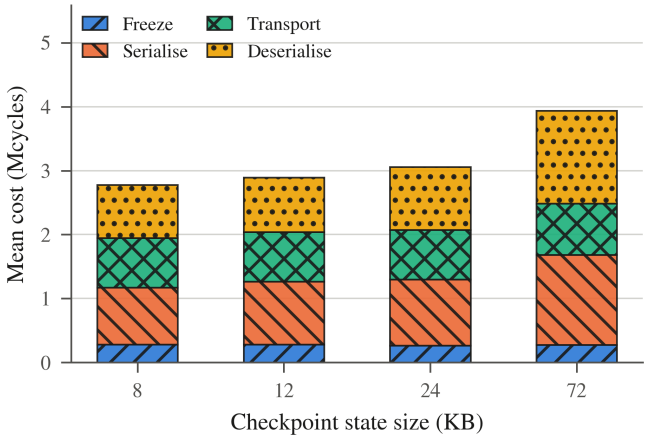


Fig. 4. Contribution of each phase to total migration cost, kernel-resident build. Serialise and deserialise absorb the growth; freeze and transport are flat.

C. Scaling with state size

Fig. 5 shows total latency against checkpoint size. The two builds track each other across the whole range, so the placement result of Section V-A is not an artefact of one operating point. The lines cross between 24 and 72 KB, which is well within the confidence intervals of both and should not be read as one build overtaking the other.

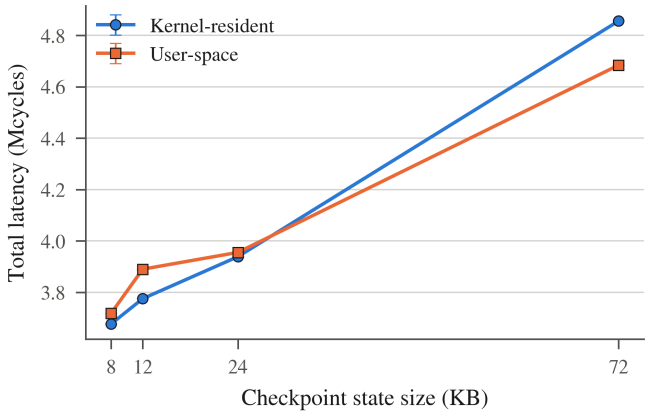


Fig. 5. Total latency against checkpoint size, with 95 per cent confidence intervals. Both builds scale with the state moved, and neither is consistently faster.

D. In-kernel code size

The security argument for a microkernel is an argument about how much code must be trusted. Fig. 6 compares the in-kernel code attributable to migration in three systems. The Mach figure is quoted from [3]; the Amoeba and our own figures are measured with wc -l on the respective source trees.

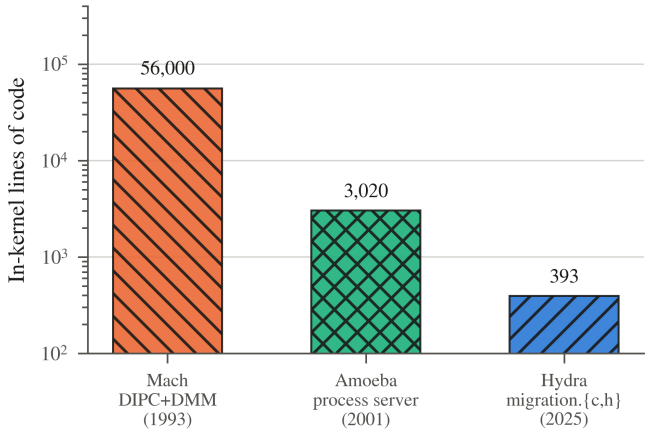


Fig. 6. In-kernel code attributable to process migration. Mach from [3]; Amoeba and this work measured from source. Note the logarithmic scale.

A second observation follows from reading the Amoeba source rather than the Amoeba literature. Although it is described throughout as a microkernel, its kernel tree contains fourteen in-kernel servers totalling 16,723 lines, among them process management, a terminal layer, a virtual disk server and an IP stack. The process server is the relevant one: it implements a checkpoint operation and support for immigrating threads, so Amoeba's migration really is kernel-resident, and is the direct functional counterpart of the code we count. Comparing the same function in both systems gives a factor of 7.7; comparing the RPC and IPC subsystems gives 8.3.

We deliberately rest no claim on whole-kernel size. Amoeba's kernel is 115,824 lines against our 5,558, but most of that gap is scope rather than discipline: Amoeba supports seven device drivers, three instruction set architectures, a

TCP/IP stack and a tape server, none of which we implement. The like-for-like comparisons are the defensible ones, and they are the ones we use.

E. Round-trip cost relative to the system call floor

An argument for moving mechanism out of the kernel should account for what the resulting boundary crossings cost. Absolute times are not comparable between the two systems, which run on different instruction sets under different emulators. The ratio of a round trip to a null system call within each system is comparable, because both operations pay the same per-trap emulation overhead on the same host, so the ratio isolates what the IPC mechanism adds over a bare privilege transition. Table IV reports it.

TABLE IV. ROUND-TRIP COST RELATIVE TO THE NULL SYSTEM CALL, MEASURED ON THE SAME HOST

System	Null syscall	Round trip	Ratio
Amoeba 5.3 (i386)	0.767 µs	19.94 µs	26.0
This work (RISC-V)	4804 cyc	190229 cyc	39.7

Amoeba: 2 runs of 60 million calls; RPC 19.94 ± 0.25 µs (sd, n = 9 blocks of 50,000). This work: n = 7 boots, ratio 95% CI half-width 1.8.

Amoeba's RPC is about 1.5 times more efficient relative to its own system-call floor than ours. Three qualifications apply. The operations are not identical: Amoeba's is a request and reply between two user processes, while ours is a send and receive to a supervisor-mode server and so crosses the boundary on one side only.

More importantly, our figure includes the scheduling latency required to run the server, which falls inside the measured interval. An unknown but substantial part of the 39.7 is scheduler delay rather than IPC cost, and our scheduler is a plain round-robin. That is a property of this implementation, not of the architecture being argued for, but it is a real cost and we do not discount it.

Third, the measurement proved highly sensitive to instrumentation. Our IPC path originally emitted a trace line over the serial port for every message sent and received. Because that write is synchronous it dominated the measurement completely: a round trip cost 7.5 million cycles with tracing enabled against 252,000 with it disabled, a factor of thirty. The figures in Table IV were taken with per-message tracing compiled out. We record the number because it is a trap that any similar measurement can fall into.

It is worth asking why Amoeba is the faster of the two, because the answer is not accidental. Amoeba's RPC was the product of sustained optimisation against exactly this metric, carried in a kernel-resident path with a purpose-built protocol, and Fig. 1 shows it leading its generation. Ours is a round-robin-scheduled message queue in a teaching-scale kernel that has not been tuned for latency at all. The comparison is therefore unflattering by construction, and we report it for that reason: it establishes that the placement result of Section V-A is not an artefact of an IPC path so slow that nothing could distinguish the two builds. Even against a system whose IPC is 1.5 times more efficient relative to its own floor, the cost of a crossing remains three orders of magnitude below the cost of a migration.

F. Cross-node migration

Eight of fifteen attempted cross-node migrations completed end to end. The seven failures were fragment loss in the unreliable multicast transport, not faults in checkpoint or restore: in every failed run the source fragmented successfully and the destination timed out waiting for a missing frame. Table V gives the costs of the successful runs.

TABLE V. CROSS-NODE MIGRATION COST, CYCLES, MEAN ± 95% CI HALF-WIDTH, N = 8

Phase	Mean cycles
Source fragmentation	1,086,675 ± 181,428
Destination reassembly	3,007,310 ± 881,189
Destination restore	1,129,470 ± 66,836

Real network transport adds approximately 4.1 Mcycles over the local baseline at 8 KB of state, and the confidence intervals are much wider than in the local experiment, as one expects of a best-effort transport. The result we take from this is qualitative: the checkpoint record is a sufficient description of a task to reconstruct it on a different machine, and the transport cost is separable from the mechanism cost.

G. Fault tolerance from the same primitives

Migration and fault recovery are the same mechanism used for different reasons: both checkpoint a task and restore it. The 1999 survey [3] identifies fault resilience as one of the applications that would justify migration, and systems of that era generally needed a separate subsystem to provide it — a checkpoint library distinct from the migration machinery, each of substantial size. If the two are really one mechanism, one pair of system calls should serve both.

A demonstration confirms this. A task incrementing a counter is spawned and allowed to run; a checkpoint is taken; the task runs further; the task is then killed to simulate a fault; the failure is detected by observing that the counter has stopped advancing; and the task is restored from the saved record. Fig. 7 shows the sequence.

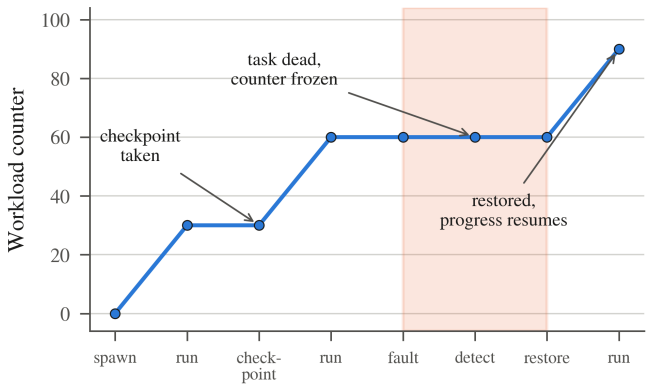


Fig. 7. Workload progress across the checkpoint, fault and recovery sequence. The counter advances, freezes while the task is dead (shaded), and advances again once the task is restored from the checkpoint taken earlier.

The checkpoint was 8,340 bytes and was taken with the counter at 30; the task reached 60 before being killed; the counter remained at 60 for the duration of the fault; and after restore the workload advanced to 90. Checkpointing cost 2,848,714 cycles and restoring 1,433,502 cycles, so recovery is roughly half the price of the save — which is what one expects, since restore writes a task structure and maps a stack while checkpoint must also walk and serialise the source.

One limitation should be stated precisely. The counter in this demonstration is a global variable rather than task-private state, so the figure shows the workload resuming execution, not the counter rolling back to its checkpointed value. What the demonstration establishes is that the restored task is a live, scheduled task that continues to make progress; a full rollback-recovery argument would require the workload's state to live inside the checkpointed region, which our bounded process model supports but this particular demonstration does not exercise.

The relevance to Section V-D is direct. Because fault recovery and migration share the same two system calls, the in-kernel cost of providing both is the 393 lines counted there, not that figure plus a separate checkpointing subsystem.

VI. Discussion

A. The performance premise has gone

The 1993 verdict rested on a cost asymmetry: crossing the kernel boundary was expensive, so a design that crossed it repeatedly lost to one that did not. Our measurements do not contradict that reasoning; they remove its premise. On this kernel the asymmetry is absent, and with it the performance case for absorbing distribution mechanism into the kernel. We are not claiming to have beaten Mach at its own benchmark on its own hardware. We are claiming that the 1993 measurement is no longer informative for a present-day architectural decision.

What remains is the code-size argument, and it now points the other way. If two placements perform the same, the one requiring 393 lines of trusted code rather than tens of thousands is preferable on the grounds the modern microkernel literature treats as decisive [6], [7]. The Amoeba comparison strengthens this: the system that best represented the kernel-resident philosophy, and that led its generation on IPC performance, still needed 3,020 lines inside the kernel for the same function.

B. The non-technical barriers, revisited

The 1999 survey [3] attributes migration's failure to reach production as much to non-technical barriers as to performance: no compelling application, no supporting infrastructure, resistance from users and administrators, and the availability of simpler alternatives such as remote execution. Three of those four have weakened materially. Checkpoint and restore is now infrastructure rather than research, shipped in container platforms [19] and in automotive operating systems at scale [21]; the applications that motivate it, long-running jobs holding accelerator state, did not exist in 1999; and administrator resistance has largely dissolved into the normal operation of orchestrated clusters, where rescheduling a workload is routine.

The fourth barrier stands. For stateless request handlers, restarting elsewhere remains simpler and cheaper than migrating, and no measurement in this paper argues otherwise. The barrier has narrowed rather than fallen: it applies to a class of workload for which migration was never the right tool, and not to the class that motivates the mechanism today.

C. What we are not claiming

We are not claiming that user-space migration is faster; we measure it as indistinguishable. We are not claiming our kernel is a better operating system than Amoeba, which is a complete system with a filesystem, a network stack and multiprocessor support, none of which we implement. We are not claiming that the result generalises to kernels with a costly privilege boundary — Section VII sets out why it may not. The claim is narrow: on a contemporary RISC-V microkernel, the placement of migration mechanism is not decided by performance, and is therefore decided by trusted-code size.

VII. Threats to Validity

All cycle counts are emulator artefacts. Ratios and curve shapes survive emulation; absolute cycle counts do not, and we make no claim on them. The two systems compared in Section V-E run under different emulators on different instruction sets, which is why that comparison is expressed as a ratio rather than in seconds.

The migration benchmarks of Section V-A do not cross a hardware privilege boundary: the user-space build reaches the kernel through a function call rather than a trap, so the 1.00 ratio is a lower bound on the user-space cost. Section V-E bounds what that omission is worth, measuring a real user-to-supervisor transition and back at 4,804 cycles against a total migration cost of about 4.06 Mcycles; a migration server making even a few hundred such calls would remain well inside the confidence intervals of Table III. Replicating the comparison on a kernel with a hardware-enforced boundary, seL4 on RISC-V being the obvious candidate, is the first piece of follow-on work we would do.

The process model is bounded: fixed-size stacks, no demand paging, and no open file or socket state to carry. Systems that migrate richer processes must move more, and the serialise and deserialise phases that dominate Fig. 4 would grow accordingly; whether they grow equally in both placements is not something our data can answer. The cross-node sample is small at eight successful runs, and its transport is unreliable by construction, so Table V should be read as an order of magnitude rather than a precise cost.

Finally, both builds were measured by the same author on the same host. We mitigate the obvious risk by deriving both from one source tree, by instrumenting them identically, and by publishing the harness and the raw logs, but we have not had the comparison independently replicated.

VIII. Conclusion

We repeated a 1993 architectural experiment on a RISC-V microkernel and found its result no longer reproduces. Kernel-resident and user-space placement of process migration are statistically indistinguishable on every phase we measured, with a total-latency ratio of 1.00 at 95 per cent confidence over 60 migrations per build across four checkpoint sizes. The kernel cost of the two designs differs by more than two orders of magnitude against Mach and by a factor of 7.7 against Amoeba, the latter measured from Amoeba's own source rather than quoted.

Where the performance argument once favoured the kernel-resident design and the code-size argument was secondary, the performance argument is now neutral and the code-size argument is not. We also report that our own IPC path is the less efficient of the two systems compared, at 39.7 null system calls per round trip against Amoeba's 26.0, which qualifies the recommendation without reversing it.

Availability. The kernel, the migration implementation, the benchmark harness, the raw measurement logs, and the scripts that generate every figure and table in this paper are available as open source, together with the procedure used to boot and measure the Amoeba reference system.

. References

[1] D. Milojicic, W. Zint, A. Dangel, and P. Giese, “Task migration on top of the Mach microkernel,” in Proc. 3rd USENIX Mach Symp., 1993, pp. 273–290.

[2] D. Milojicic, F. Douglis, Y. Paindaveine, R. Wheeler, and S. Zhou, “Process migration,” ACM Comput. Surv., vol. 32, no. 3, pp. 241–299, 2000.

[3] D. Milojicic, F. Douglis, Y. Paindaveine, R. Wheeler, and S. Zhou, “Process migration,” HP Labs, Tech. Rep. HPL-1999-21, 1999.

[4] M. Accetta, R. Baron, W. Bolosky, D. Golub, R. Rashid, A. Tevanian, and M. Young, “Mach: A new kernel foundation for UNIX development,” in Proc. USENIX Summer Conf., 1986, pp. 93–112.

[5] J. Liedtke, “On µ-kernel construction,” in Proc. 15th ACM Symp. Operating Systems Principles (SOSP), 1995, pp. 237–250.

[6] G. Klein et al., “seL4: Formal verification of an OS kernel,” in Proc. 22nd ACM Symp. Operating Systems Principles (SOSP), 2009, pp. 207–220.

[7] S. Biggs, D. Lee, and G. Heiser, “The jury is in: Monolithic OS design is flawed — microkernel-based designs improve security,” in Proc. 9th Asia-Pacific Workshop on Systems (APSys), 2018, pp. 1–7.

[8] J. Gu, X. Wu, W. Li, N. Liu, Z. Mi, Y. Xia, and H. Chen, “Harmonizing performance and isolation in microkernels with efficient intra-kernel isolation and communication,” in Proc. USENIX Annu. Tech. Conf., 2020, pp. 401–417.

[9] H. Chen, Y. Du, Z. Mi, and R. Chen, “Microkernel goes general: Performance and compatibility,” in Proc. 18th USENIX Symp. Operating Systems Design and Implementation (OSDI), 2024.

[10] A. S. Tanenbaum, R. van Renesse, H. van Staveren, G. J. Sharp, S. J. Mullender, J. Jansen, and G. van Rossum, “Experiences with the Amoeba distributed operating system,” Commun. ACM, vol. 33, no. 12, pp. 46–63, 1990.

[11] S. J. Mullender, G. van Rossum, A. S. Tanenbaum, R. van Renesse, and H. van Staveren, “Amoeba — A distributed operating system for the 1990s,” IEEE Computer, vol. 23, no. 5, pp. 44–53, 1990.

[12] S. Bosse, “FSD-Amoeba 2002B: A free software distribution of the Amoeba distributed operating system,” 2002. [Online]. Available: https://sourceforge.net/projects/fsd-amoeba/

[13] A. Waterman and K. Asanovic, The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA, Version 20191213, RISC-V Foundation, 2019.

[14] F. Douglis and J. Ousterhout, “Transparent process migration: Design alternatives and the Sprite implementation,” Softw. Pract. Exper., vol. 21, no. 8, pp. 757–785, 1991.

[15] A. Barak and A. Litman, “MOS: A multicomputer distributed operating system,” Softw. Pract. Exper., vol. 15, no. 8, pp. 725–737, 1985.

[16] M. L. Powell and B. P. Miller, “Process migration in DEMOS/MP,” in Proc. 9th ACM Symp. Operating Systems Principles (SOSP), 1983, pp. 110–119.

[17] B. Walker, G. Popek, R. English, C. Kline, and G. Thiel, “The LOCUS distributed operating system,” in Proc. 9th ACM Symp. Operating Systems Principles (SOSP), 1983, pp. 49–70.

[18] D. R. Cheriton, “The V distributed system,” Commun. ACM, vol. 31, no. 3, pp. 314–333, 1988.

[19] CRIU Project, “Checkpoint/restore in userspace,” 2024. [Online]. Available: https://criu.org/

[20] F. Bellard, “QEMU, a fast and portable dynamic translator,” in Proc. USENIX Annu. Tech. Conf., FREENIX Track, 2005, pp. 41–46.

[21] BlackBerry QNX, “Software-defined vehicles: The ultimate guide,” 2024. [Online]. Available: https://blackberry.qnx.com/